



```

1 x = a - 3
2 y = a + 3
3 if x > a + 3:
4     a = a * 3
5 else :
6     x = a + 3
7 end
8 y = a - 3

```

- **gen(I)**: Conjunto das novas **expressões geradas** na instrução I.
- **kill(I)**: Conjunto das **expressões eliminadas** na instrução I, ou seja a partir de I a expressão não está mais disponível para reutilização pois seus operandos foram alterados.

Instrução I	$gen(I)$	$kill(I)$

(c) Use o algoritmo iterativo visto em aula para preencher os conjuntos $IN(I)$ e $OUT(I)$.

- **$IN(I)$** : Conjunto das expressões disponíveis na **entrada** de I .
- **$OUT(I)$** : Conjunto das expressões disponíveis na **saída** de I .

Instrução I	$IN(I)$	$OUT(I)$

(d) Por fim, discuta: como a análise de **Expressões Disponíveis** pode ser utilizada para otimizar um programa? Quais outras otimizações se beneficiam da análise de expressões disponíveis?

2. **Grafo de Fluxo.** Aqui está um programa simples que lê dois números e imprime o maior divisor comum deles, desde que ambos os números sejam positivos.

```

1 x = read ();
2 y = read ();
3 while (x != y) {
4     if (x > y) {
5         x = x - y;
6     } else { /* y > x */
7         y = y - x;
8     }
9 }
10 print (x);

```

(a) Desenhe o CFG no nível dos blocos básicos. (bloco básico é um trecho de código que só possui 1 ponto de entrada e 1 ponto de saída).

(b) Defina o que é a forma SSA (*Static Single Assignment* - Atribuição estática única).

(c) Explique o desafio de transformar o CFG da letra (a) em um CFG na forma SSA.

3. **Coletor de Lixo.** Algumas linguagens de programação executam gerenciamento de memória automático ("coleta de lixo") durante a execução do código.

(a) Explique por quê isso é necessário. (1pt)

(b) Em determinado ponto no programa, o que pode ser considerado Lixo? Como podemos encontrar Lixo na memória? (3pts)

(c) Cite pelo menos duas desvantagens da estratégia "Contagem de Referências" (*reference counting*) para coleta de lixo. (2pts)

4. **Memória em C.** Em linguagens compiladas como C e C++, não há gerenciamento de memória dinâmico. Explique as implicações positivas e negativas em linguagens de programação com essa característica.

5. **Otimização.** Aplique todas as otimizações que você conhece nos programas abaixo. Liste a ordem em que você as aplicou.

(a) Programa 1:

```
1 a = x - y
2 if (a > 3) {
3     b = a + z
4     c = x - y
5 } else {
6     b = a + z
7     a = a * b
8 }
9 d = x - y
10 b = b / d
11 print(a * b)
```

(b) Programa 2:

```
1 b := 2
2 i := 1
3 do {
4     a := b + 1
5     c := 2
6     if (i % 2) == 0 {
7         d := a + d
8         e := 1 + d
9     } else {
10        d := - c
11        f := 1 + a
12    }
13    i = i + 1
14 } while (a < 2)
```